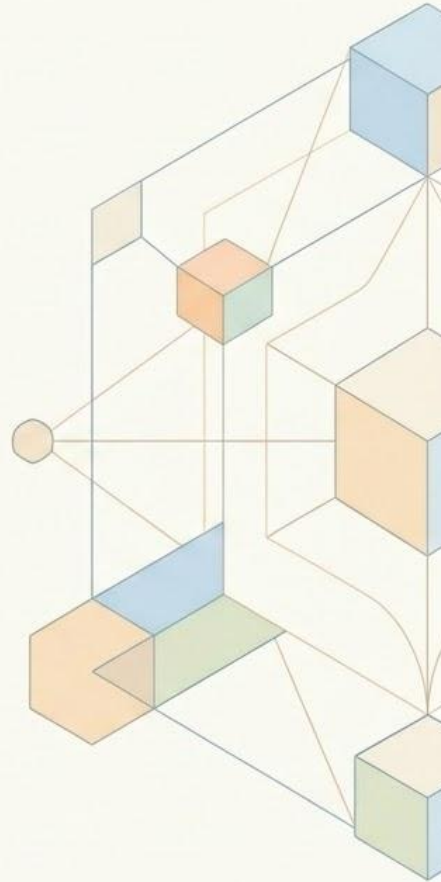
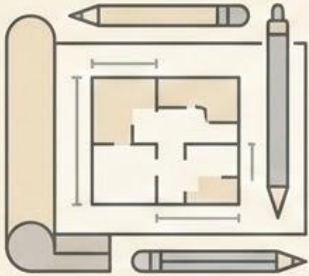


Object-Oriented Programming in Python

Classes, Objects, Inheritance & Polymorphism

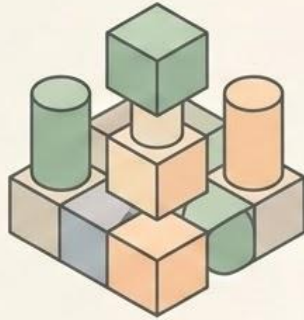


What is Object-Oriented Programming?



Blueprint

- Organize code



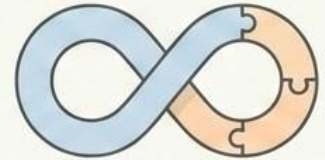
Building Blocks

- Model real-world things



Real World

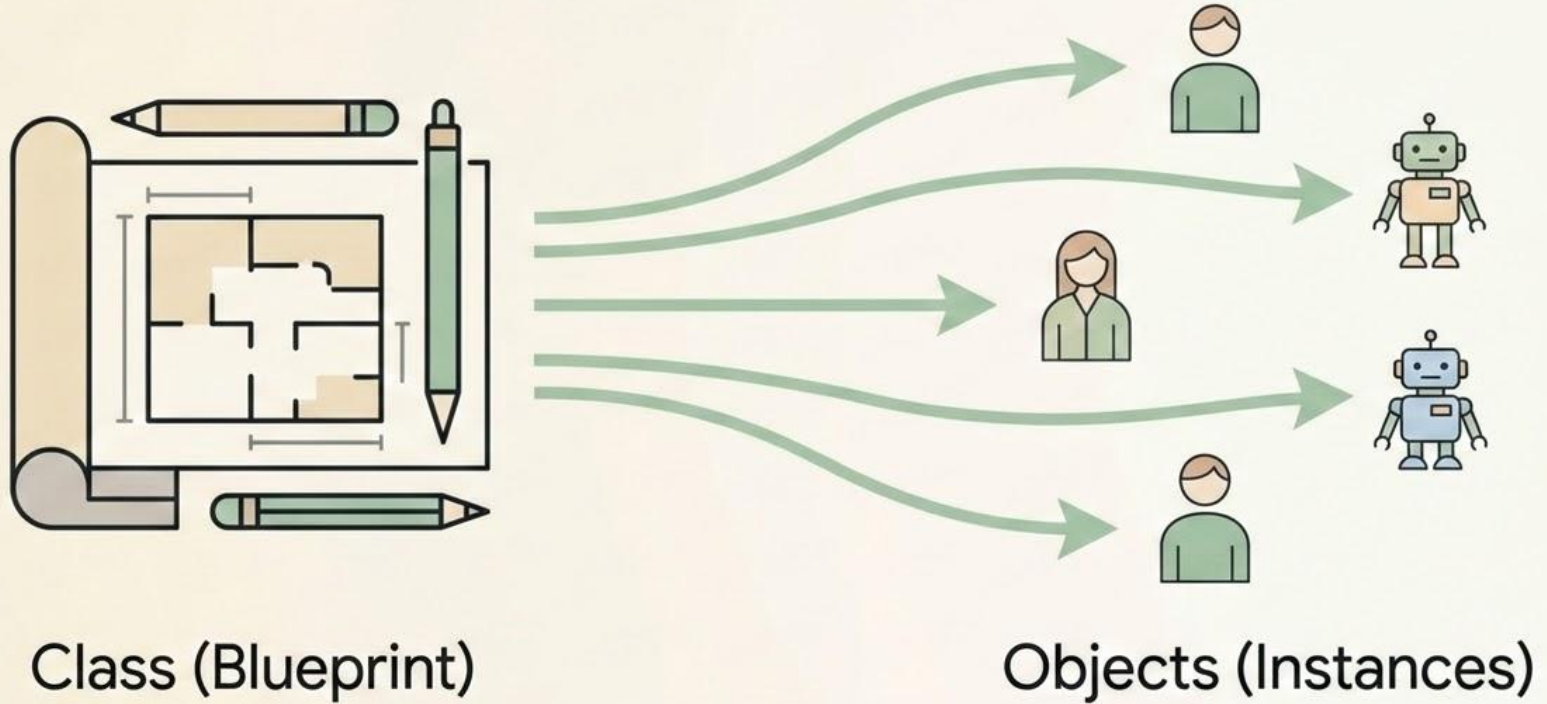
- Promote reuse



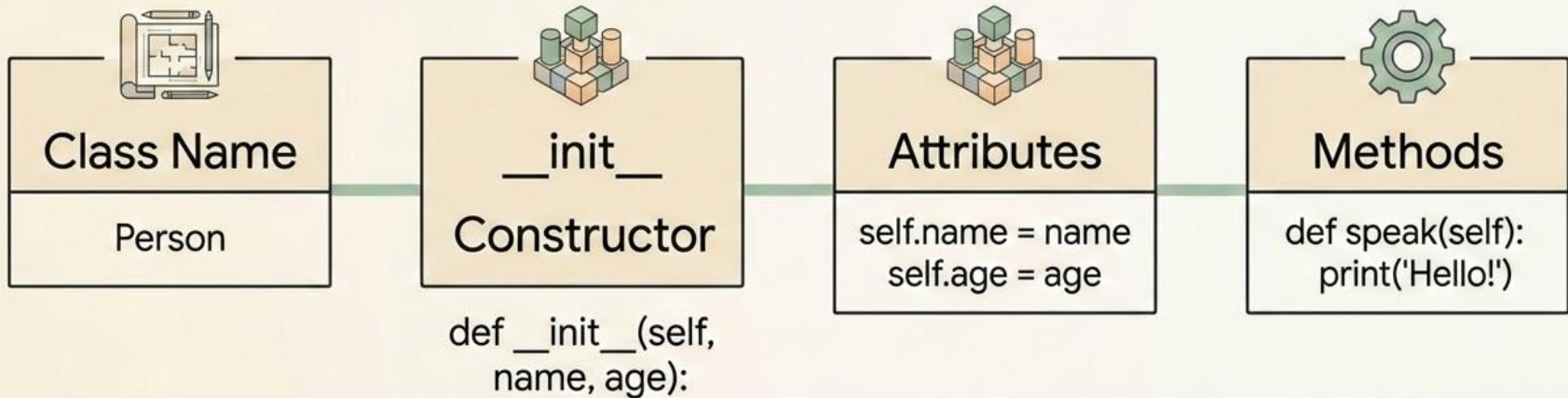
Reusability

- Improve maintainability

Classes vs Objects



Anatomy of a Class

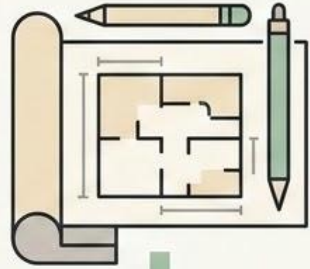


Creating Objects (Instances)

```
# Instantiate p1
p1 = Person("Alice", 30)

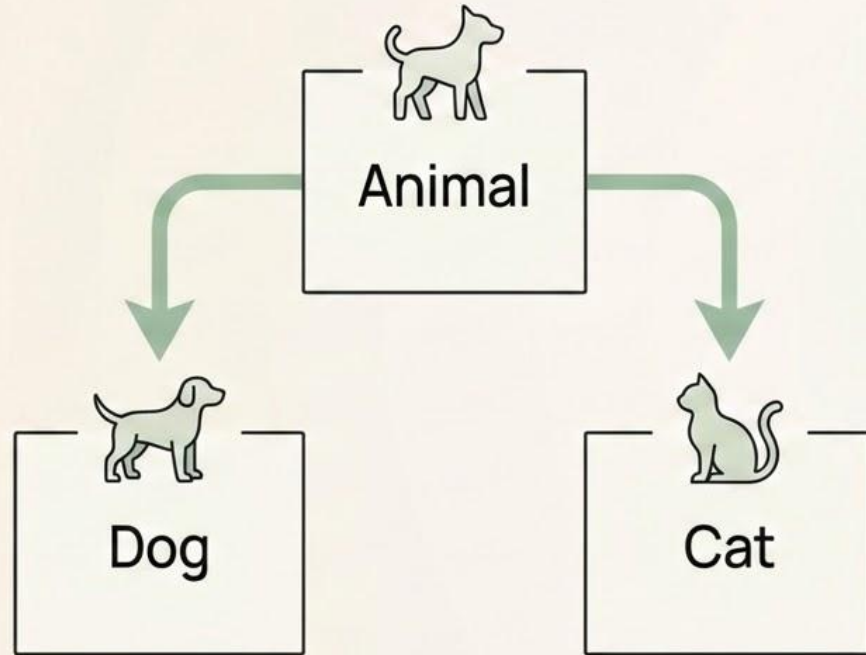
# Instantiate p2
p2 = Person("Bob", 25)

# Print attributes
print(f"p1: {p1.name}, {p1.age}")
print(f"p2: {p2.name}, {p2.age}")
```



p1 = Person('Alice', 30)

Inheritance - Code Reuse



Child classes inherit attributes and methods from parent

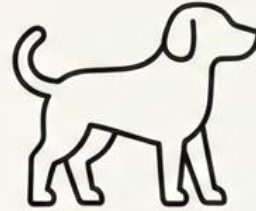
Method Overriding

Parent class



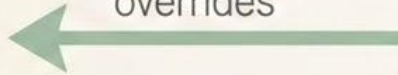
`Animal.speak()`
-> "Some sound"

Child class

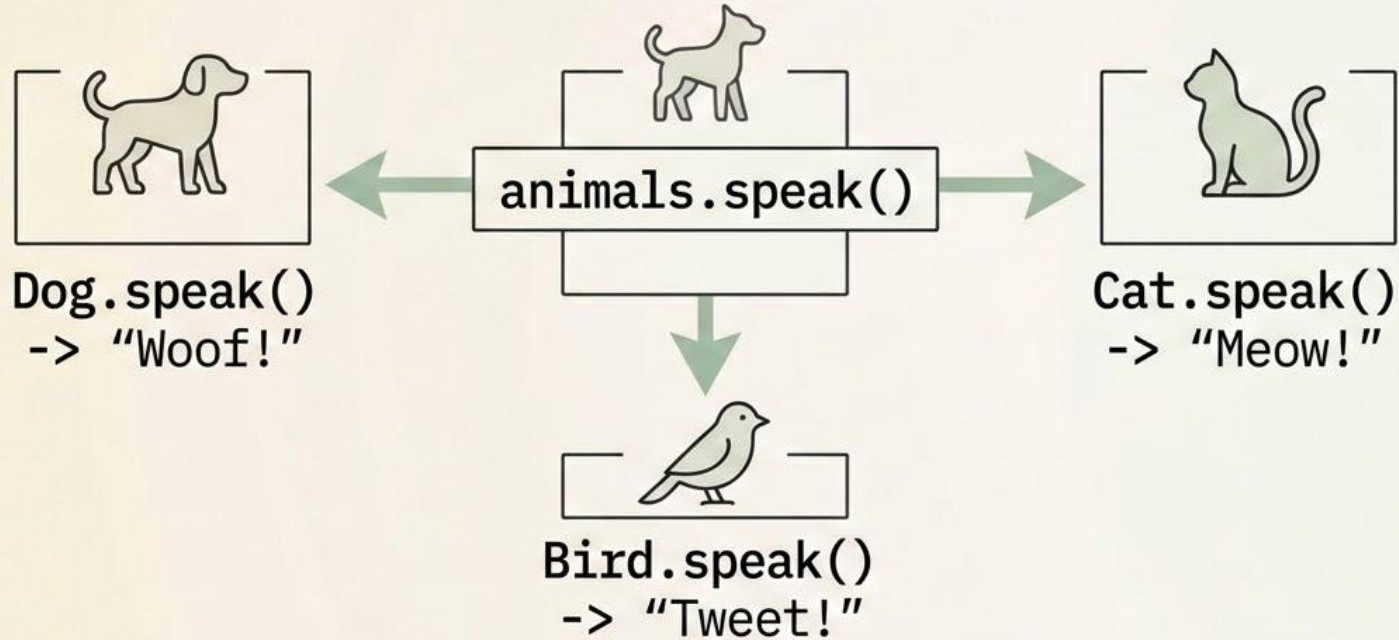


`Dog.speak()`
-> "Woof!"

overrides

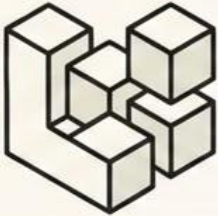


Polymorphism - Many Forms

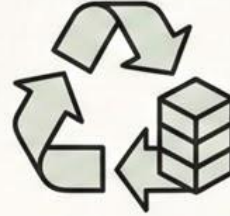


Same interface, different behaviors

Why Use OOP?



Modularity
Easier code
partitioning



Reusability
Less code
repetition



Maintainability
Simpler updates
and fixes

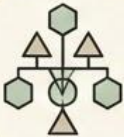


**Real-world
Modeling**
Conceptualizing
complex systems

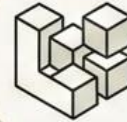
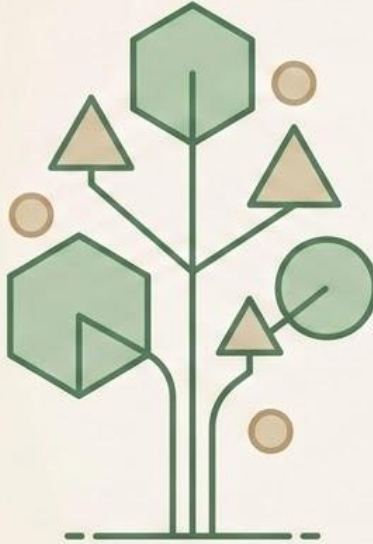
Key Takeaways



Classes =
Blueprints



Inheritance
= Reuse



Objects =
Real instances



Polymorphism
= Flexibility

Write better, cleaner, reusable Python code